

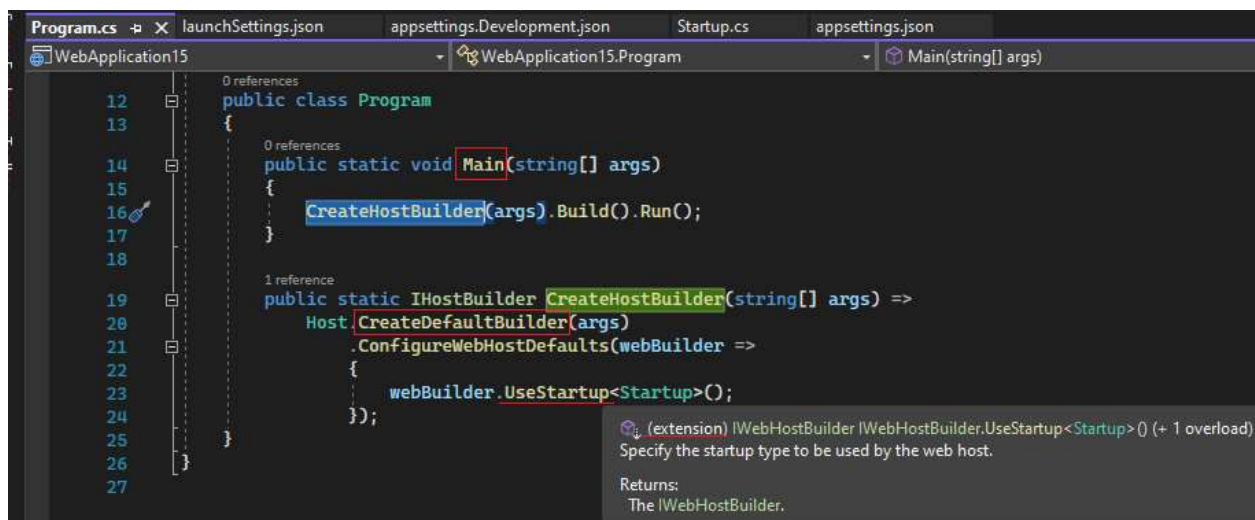
Коментари в .json файловете са трудни: <https://stackoverflow.com/questions/244777/can-comments-be-used-in-json/7901053#7901053>

Middleware

Middleware – междинен софтуер, доста широк термин; парче софтуер, което може да обработи http-заявка или отговори на даден компонент със специално предназначение, например за аутентификация на потребител, друг междинен софтуер за прихващане на грешки, друг да подава статични файлове като JavaScript-файлове, CSS-файлове, картинки и т.н. Използваме междинен софтуер за настройване на съобщителния канал pipeline, който определя как да се обработват заявките.

.Net проектът стартира Program.cs, с отправна точка главния Main-метод, който извиква клас Startup, дефиниран във файла със същото име Startup.cs.

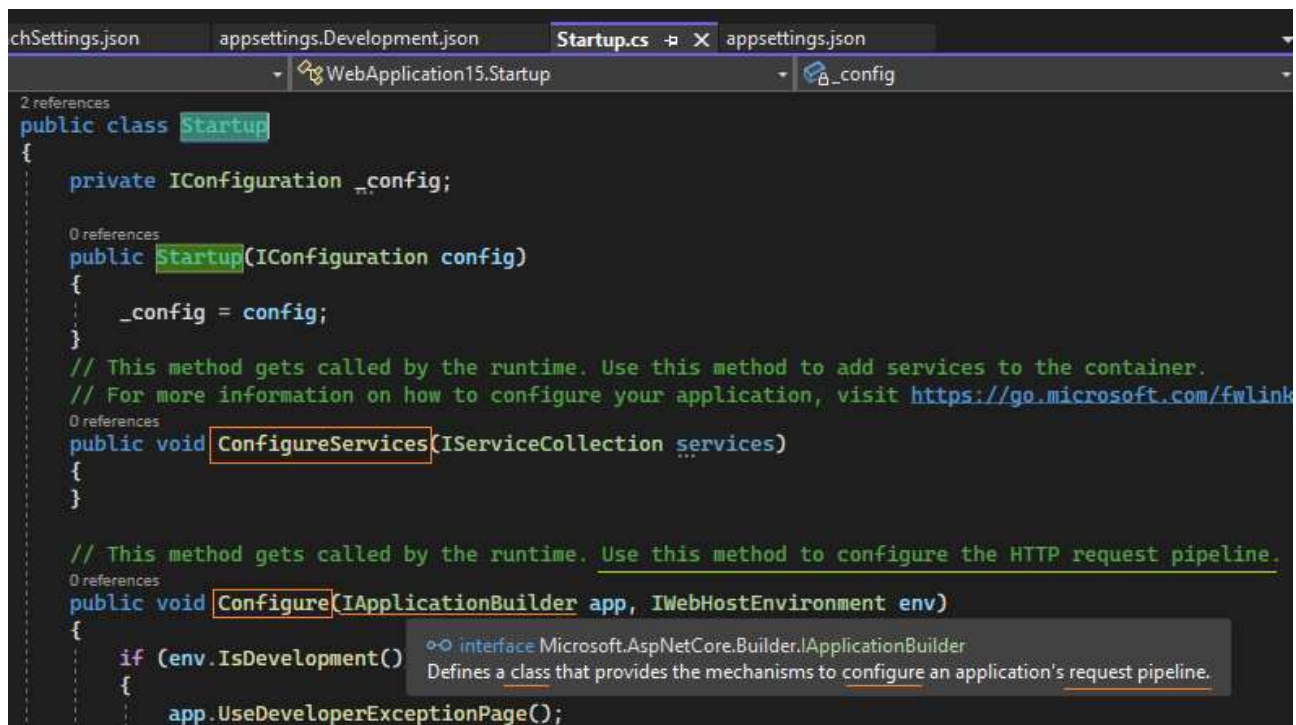
Main-методът извиква CreateDefaultBuilder, който от своя страна настройва WebHost по подразбиране. За да настроим всичко това, използваме споменатият клас <Startup>, извикан чрез .UseStartup-разширението.



```
12 public class Program
13 {
14     public static void Main(string[] args)
15     {
16         CreateHostBuilder(args).Build().Run();
17     }
18
19     public static IWebHostBuilder CreateHostBuilder(string[] args) =>
20     {
21         Host.CreateDefaultBuilder(args)
22             .ConfigureWebHostDefaults(webBuilder =>
23             {
24                 webBuilder.UseStartup<Startup>();
25             });
26     }
27 }
```

Във файла Startup.cs имаме клас с неговото име Startup, който управлява 2 метода: ConfigureServices + Configure.

Configure-методът конфигурира обработката на заявките по съобщителния канал pipeline.



```
chSettings.json  appsettings.Development.json  Startup.cs  appsettings.json
WebApplication15.Startup
2 references
public class Startup
{
    private IConfiguration _config;

    0 references
    public Startup(IConfiguration config)
    {
        _config = config;
    }

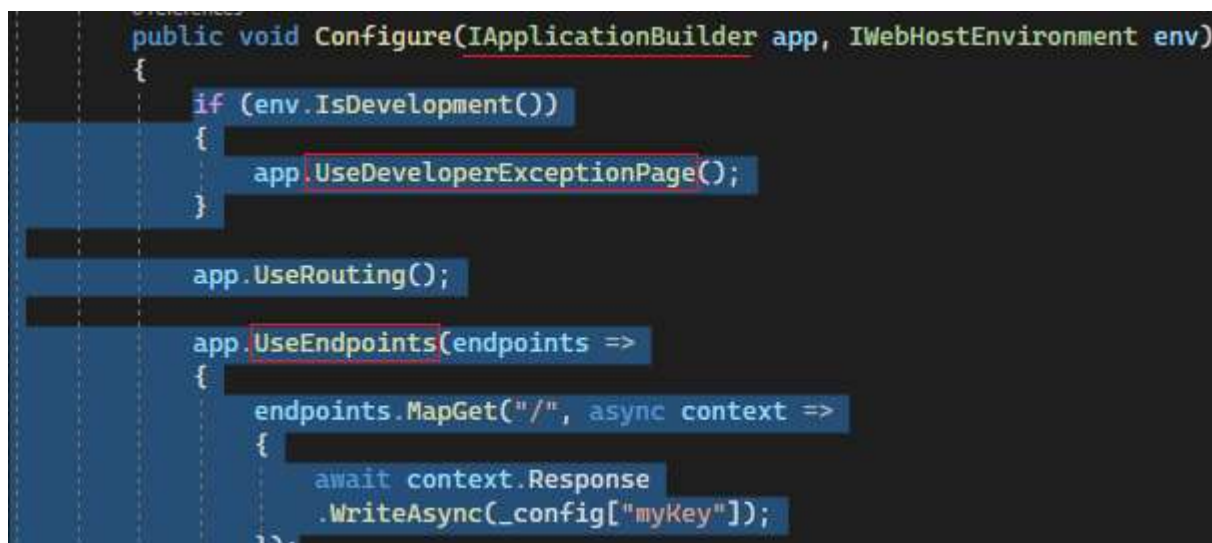
    // This method gets called by the runtime. Use this method to add services to the container.
    // For more information on how to configure your application, visit https://go.microsoft.com/fwlink
    0 references
    public void ConfigureServices(IServiceCollection services)
    {
    }

    // This method gets called by the runtime. Use this method to configure the HTTP request pipeline.
    0 references
    public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
    {
        if (env.IsDevelopment())
        {
            app.UseDeveloperExceptionPage();
        }
    }
}
```

interface Microsoft.AspNetCore.Builder.IApplicationBuilder
Defines a class that provides the mechanisms to configure an application's request pipeline.

Кодът в този Configure-метод е генериран автоматично от Empty-project template. Той задава много проста обработка на заявките по съобщителния канал със само два middleware:

1) UseDeveloperExceptionPage и 2) UseEndpoints, който във 2.2 е Run в този IApplicationBuilder-интерфейс:



```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response
                .WriteAsync(_config["myKey"]);
        });
    });
}
```

* Middleware компонентите имат достъп до входящи заявки и изходящи отговори, така че сливащият компонент прихваща входящата заявка, придвижва я по съобщителния канал, където я обработва следващото парче код.

Middleware in ASP.NET Core



Middleware in ASP.NET Core

- Has access to both Request and Response
- May simply pass the Request to next Middleware
- May process and then pass the Request to next Middleware
- May handle the Request and short-circuit the pipeline
- May process the outgoing Response
- Middlewares are executed in the order they are added

Например Logging-middleware е първото парче код в съобщителния канал на нашата апликация, така че, когато заявката пристигне от уеб-сървъра, в това междинно парче код регистрира времето на получаване и го прехвърля към следващото междинно парче код (в нашия случай до статичните файлове за по-нататъчна обработка).

* Също може да откаже по-нататъчна обработка, ако има условията за това, което се нарича short-circuiting (къса верига) по съобщителния канал pipeline. Това скъсване често е предпочитано, защото избягва излишна обработка.

Например, ако заявката е за статичен файл (картинка, CSS, JS), ще се извика късата верига и приключи, без да се предаде за обработка към MVC кодовете.

* Може да откаже изцяло каквато и да е обработка, ако издадем заявка към localhost за намаляване на служителите, ако основно искаме да извлечем списъка на всички служители, така че, ако тази заявка пристигне от уеб-сървъра, първото парче код в нашия съобщителен канал, което е Logging, ще отчете времето на пристигане на заявката, ще маркира, че това не е статичен файл, който няма нужда от допълнителна обработка, така че ще прескочи кода, предназначен за статични файлове и ще го насочи за допълнителна обработка към активния MVC model-view-controller, който да произведе HTTP-отговор. В този случай съобщителния канал превключва обратно отговора към кода за обработка на статичните файлове, който от своя страна няма интерес да ги обработва по-нататък, така че просто оставя отговора да премине към следващото парче код, което е Logging, което просто отчита времето на връщане на отговора на обработената заявка.

Важно е да се запомни, че Logging-кода отчита двете времена – на входящата заявка и отговорът от обработката, така че това може да се използва за изчисление на общото време, необходимо за услугата и после този отговор да се предостави на уеб-сървъра, който от своя страна да върне отговор към клиента, който е поискал това.

* Поради всичко това е важно парчетата код да се извикват едно друго в правилна последователност. Тези парчета код са достъпни като NuGet-packages, което означава, че ъпдейтите са NuGet-парчета код, позволяващи ни да обновяваме всяко парче междинен код поотделно. В зависимост от изискванията на приложението, ние можем да добавяме повече или по-малко парчета междинен код към съобщителния канал за обработка на заявки. Например: ако изграждаме просто уеб-приложение с малко статични .html страници и картинки, тогава съобщителният канал може да съдържа само междинен код за обработката им. От друга страна, ако изграждаме защитено с данни уеб-приложение, ще имаме нужда от няколко междинни кодови компонента за обработки на статични файлове, ауторизации, лични данни, MVC.

* Имаме пълен контрол върху съобщителния канал. Това също означава, че от гледна точка на паметта и производителността, че ще се обработват само междинните парчета код, необходими за заявките на приложението. Накратко може да обобщим, че само всяко от ангажираните междинни парчета код има достъп едновременно до входящите заявки и изходящи отговори. Междинният код може просто да прехвърли обработката до следващото парче код, може да извърши някаква обработка и тогава да го предаде нататък, или пък изцяло да откаже обработка по къса верига. Парчетата междинен код се изпълняват в последователността, която е заложена при създаването им.

Configure request processing pipeline

```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }

    app.UseRouting();

    app.UseEndpoints(endpoints =>
    {
        endpoints.MapGet("/", async context =>
        {
            await context.Response
                .WriteAsync(_config["myKey"]);
        });
    });
}
```

Имаме междинен код методът `UseDeveloperExceptionPage`, който ще се изпълни, ако имаме изключение и ако `Environment = IsDevelopment ("ASPNETCORE_ENVIRONMENT": "Development")` от `LaunchSettings.json`

Другият метод, който имаме `UseEndpoints` може само да изведе съобщение на екрана, използвайки `.Response` обекта. Към момента, това е парчето междинен код, което отговаря на почти всички заявки, независимо каква е частта от заявката или изискванията към нея, всичко се

обработка от `.Response` и отговорът, който получаваме, е съобщението, което изписваме към обекта `_config`.

Нека опростим нещата със съобщение „Hello world” вместо ключа `myKey`.

Демонстрацията е, че всяка заявка, която идва независимо откъде, се обработва от това междинното парче код `middleware`, което се намира в `startup.cs`

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello World!");
    });
});
```

Към този момент, програмата няма възможности да обработва статични файлове.

Метод `UseEndpoints` (старо име `Run`) е изходната точка с резултата от нашата заявка в `html`-визуализиран вид.

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello World!");
    });
});
```

(extension) `ApplicationBuilder` `ApplicationBuilder.UseEndpoints(Action<Microsoft.AspNetCore.Routing.IEndpointRouteBuilder> configure)`
Adds a `Microsoft.AspNetCore.Routing.EndpointMiddleware` middleware to the specified `ApplicationBuilder` with the `Microsoft.AspNetCore.Routing.EndpointDataSource` instances built from configured `Microsoft.AspNetCore.Routing.IEndpointRouteBuilder`. The `Microsoft.AspNetCore.Routing.EndpointMiddleware` will execute the `Endpoint` associated with the current request.

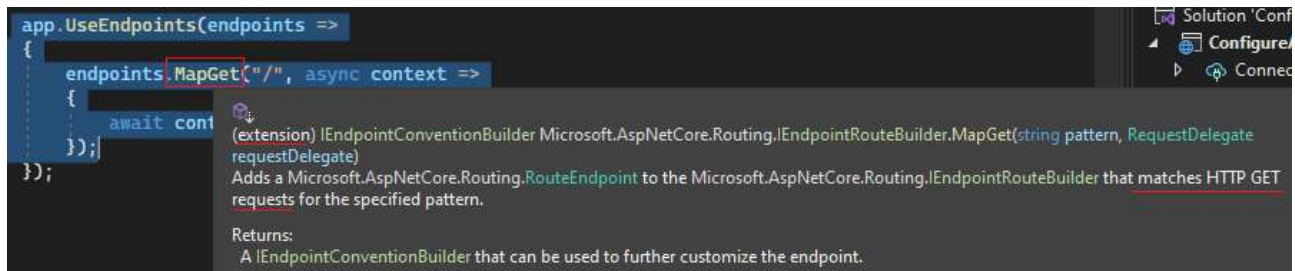
A call to `EndpointRoutingApplicationBuilderExtensions.UseEndpoints(IApplicationBuilder, Action<Microsoft.AspNetCore.Routing.IEndpointRouteBuilder>)` must be preceded by a call to `EndpointRoutingApplicationBuilderExtensions.UseRouting(IApplicationBuilder)` for the same `ApplicationBuilder` instance.

The `Microsoft.AspNetCore.Routing.EndpointRoutingMiddleware` defines a point in the middleware pipeline where routing decisions are made, and an `Endpoint` is associated with the `HttpContext`. The `Microsoft.AspNetCore.Routing.EndpointMiddleware` defines a point in the middleware pipeline where the current `Endpoint` is executed. Middleware between the `Microsoft.AspNetCore.Routing.EndpointRoutingMiddleware` and `Microsoft.AspNetCore.Routing.EndpointMiddleware` may observe or change the `Endpoint` associated with the `HttpContext`.

Returns:
A reference to this instance after the operation has completed.

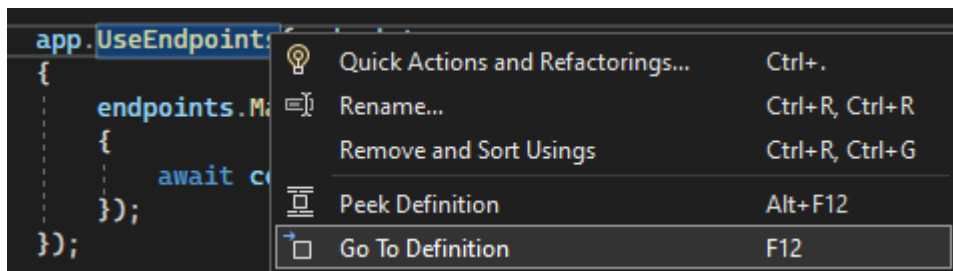
```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello World!");
    });
});
```

Параметърът е `RequestDelegate` – пратеник, който приема request HTTP context object като параметър.

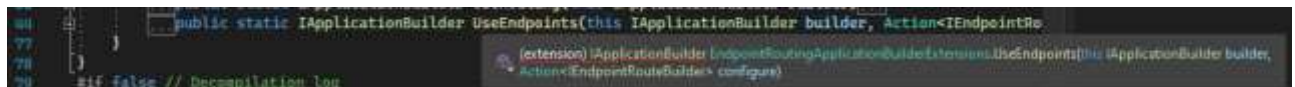


```
public delegate Task RequestDelegate(HttpContext context);
```

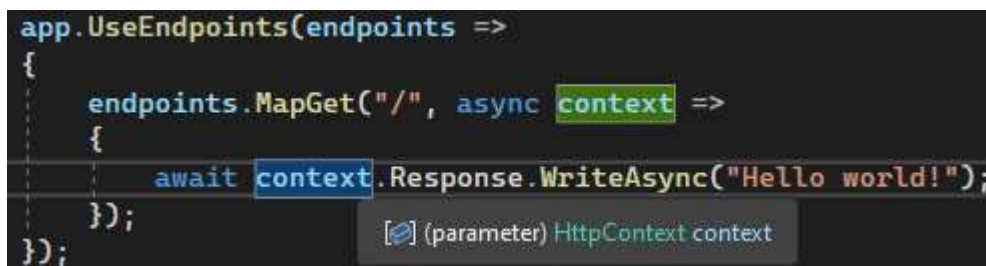
Нека разгледаме зададената дефиниция на UseEndpoints ->д.б.->Go To Definition



Резултат:



Дефинициите (Go to Definition F12) са в контекстното меню с подпроцорци с дясно подреждане и затваряне, съответно там не са позволени редакции за нов ред с Enter и има много свити описателни коментари.



С нов д.б.->Go To Definition виждаме, че context object дава възможност за входящ и изходящ контрол на данните от заявките през http чрез .Response – property.

```

namespace Microsoft.AspNetCore.Http
{
    ...public abstract class HttpContext
    {
        ...public abstract IFeatureCollection Features...
        //
        // Summary:
        //     Gets the Microsoft.AspNetCore.Http.HttpRequest object for this request.
        public abstract HttpRequest Request...
        //
        // Summary:
        //     Gets the Microsoft.AspNetCore.Http.HttpResponse object for this request.
        public abstract HttpResponse Response...
    }
}

```

Към момента, това е анонимен метод, използващ Lambda expression.

Lambda expressions (C# reference)

Article • 01/25/2022 • 15 minutes to read • 5 contributors



You use a *lambda expression* to create an anonymous function. Use the [lambda declaration operator =>](#) to separate the lambda's parameter list from its body. A lambda expression can be of any of the following two forms:

- [Expression lambda](#) that has an expression as its body:

```

C# Copy

(input-parameters) => expression

```

- [Statement lambda](#) that has a statement block as its body:

```

C# Copy

(input-parameters) => { <sequence-of-statements> }

```

To create a lambda expression, you specify input parameters (if any) on the left side of the lambda operator and an expression or a statement block on the other side.

Нека тестваме поредността на съобщителния канал.

За да изведем съобщение на екрана, използвахме досега UseEndpoints, т.е. крайната точка на програмата. Тя може да бъде само и единствено една. Няма как да копираме целия метод и да го

сложим 2 пъти. Той е край. За да тестваме, обаче, средните слоеве, можем да използваме преди него просто Use, който също да изведе съобщение на екрана.

```
app.UseRouting();

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-st Middleware!");
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});
```

```
app.Use(async (context, next) =>
```

```
{
```

```
    await context.Response.WriteAsync("Hello from 1-st Middleware!");
```

```
});
```

UseRouting е адресирането – първо да се изписва името на домейна (в нашия случай localhost:5000), после името на папката и файла (контролер – действие / Controller-> Action) и т.н. включително стойности на самите заявки. Ако се закоментира, няма да може да адресира правилно действията в браузъра и да намира директориите и ще хвърли грешка.

```
// app.UseRouting();

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-st Middleware!");
});

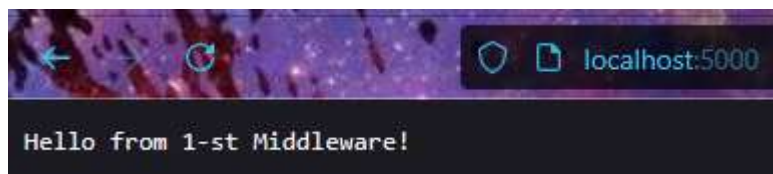
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});
```




```
app.UseRouting();

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-st Middleware!");
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        // await context.Response.WriteAsync("Hello world!");
        // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().Name);
        // await context.Response.WriteAsync(_config["MyKey"]);
        await context.Response.WriteAsync("Hello from 2-nd Middleware!");
    });
});
```



Т.е. виждаме, че Use изпреварва и дори изобщо не стига до UseEndpoints. За да продължи към следващия слой от времевата линия pipeline, ние трябва да му кажем с next- delegate със синтаксис `await next();`

```

app.UseRouting();

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello from 1-st Middleware!");
    await next();
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        // await context.Response.WriteAsync("Hello world!");
        // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().Name);
        // await context.Response.WriteAsync(_config["MyKey"]);
        await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
    });
});

```



Това са демонстрации на междинните слоеве Middleware components по времевата линия pipeline.

Когато имаме ъпдейти, които не присъстват като middleware components in Pipeline, те са под формата на NuGet packages.

Terminal middleware – последен, не може да се повтаря.

Все едно имаме:

```

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Hello 1");
    await next();
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync(" + Hello 2");
    });
});

```

За да усложним нещата, нека видим логване, което се извиква от Main() от Program.cs, а той извиква CreateHostBuilder

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs>



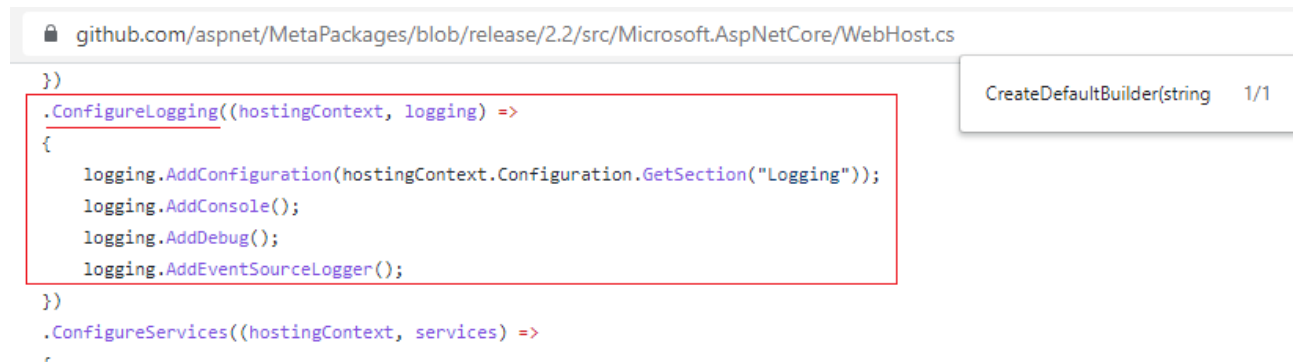
```
/// <returns>The initialized <see cref="IWebHostBuilder"/>.</returns>
public static IWebHostBuilder CreateDefaultBuilder(string[] args)
{
    var builder = new WebHostBuilder();

    if (string.IsNullOrEmpty(builder.GetSetting(WebHostDefaults.ContentRootKey)))
    {
        builder.UseContentRoot(Directory.GetCurrentDirectory());
    }
    if (args != null)
    {
        builder.UseConfiguration(new ConfigurationBuilder().AddCommandLine(args).Build());
    }

    builder.UseKestrel((builderContext, options) =>
    {
        options.Configure(builderContext.Configuration.GetSection("Kestrel"));
    })
    .ConfigureAppConfiguration((hostingContext, config) =>
    {

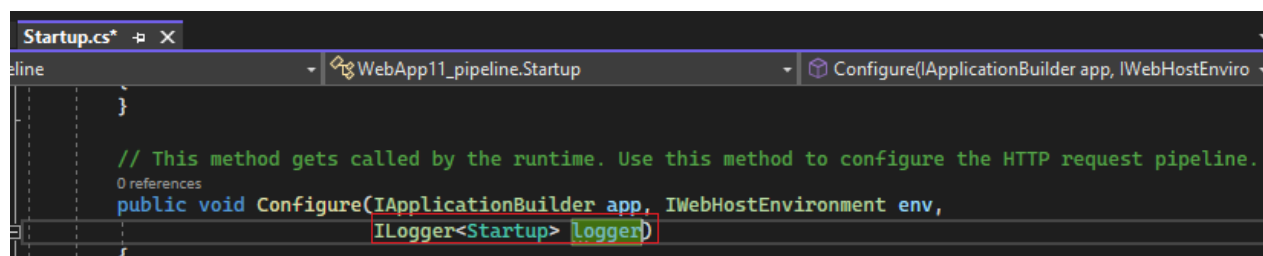
```

Виждаме и по-надолу, че конфигурира и logging



```
    })
    .ConfigureLogging((hostingContext, logging) =>
    {
        logging.AddConfiguration(hostingContext.Configuration.GetSection("Logging"));
        logging.AddConsole();
        logging.AddDebug();
        logging.AddEventSourceLogger();
    })
    .ConfigureServices((hostingContext, services) =>
    {

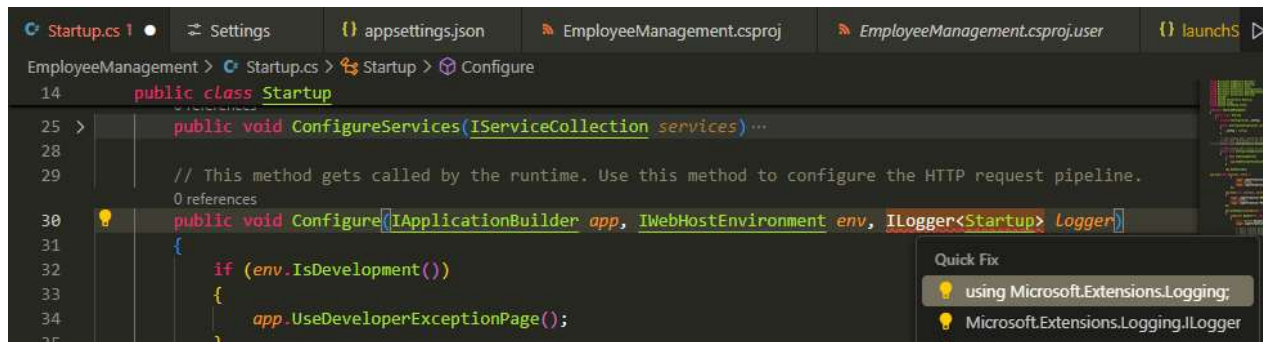
```



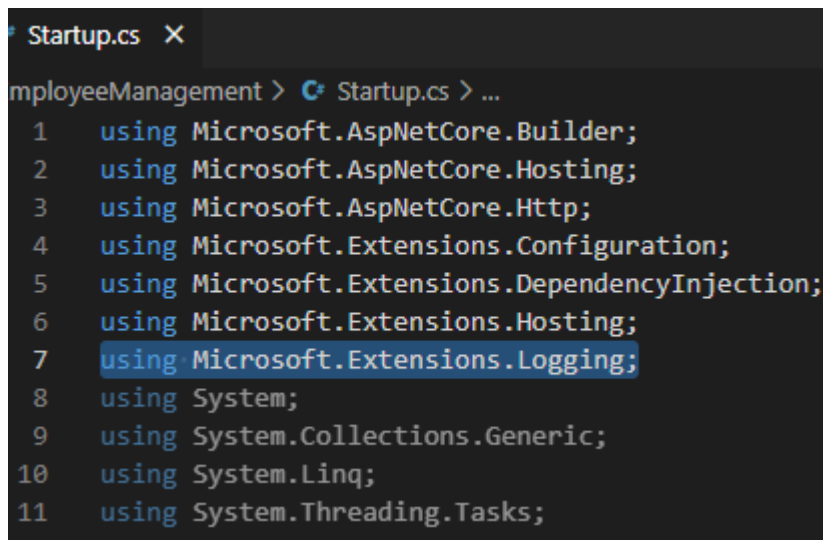
```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env,
    ILogger<Startup> logger)
{
    // This method gets called by the runtime. Use this method to configure the HTTP request pipeline.
    // ...
}
```

ILogger<Startup> logger

Като допълнение да добавим логър за клас startup с име logger и съответно с Ctrl . – namespace



using Microsoft.Extensions.Logging;



app.Use(async (context, next) =>

```
{
    logger.LogInformation("MW1: Incoming Request");
    await next();
    logger.LogInformation("MW1: Outgoing Response");
});
```

app.Use(async (context, next) =>

```
{
    logger.LogInformation("MW2: Incoming Request");
    await next();
}
```

```
    logger.LogInformation("MW2: Outgoing Response");
});
```

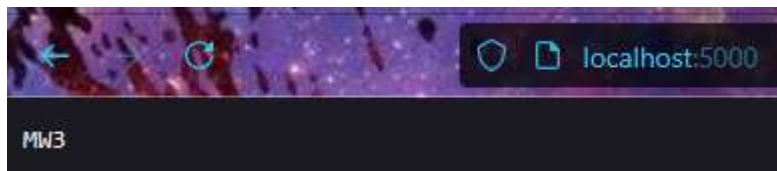
```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("MW3");
        logger.LogInformation("MW3: Request handled and response produced");
    });
});
```

```
app.UseRouting();

app.Use(async (context, next) =>
{
    // await context.Response.WriteAsync("Hello from 1-st Middleware!");
    logger.LogInformation("MW1: Incoming Request");
    await next();
    logger.LogInformation("MW1: Outgoing Response");
    // await next();
});

app.Use(async (context, next) =>
{
    logger.LogInformation("MW2: Incoming Request");
    await next();
    logger.LogInformation("MW2: Outgoing Response");
});

app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        // await context.Response.WriteAsync("Hello world!");
        // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().Name);
        // await context.Response.WriteAsync(_config["MyKey"]);
        // await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
        await context.Response.WriteAsync("MW3");
        logger.LogInformation("MW3: Request handled and response produced");
    });
});
```

```
info: Microsoft.Hosting.Lifetime[0]
  Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
  Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
  Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
  Content root path: D:\Exercises\12 class\EmployeeManagement\EmployeeManagement
info: EmployeeManagement.Startup[0]
  MW1: Incoming Request
info: EmployeeManagement.Startup[0]
  MW2: Incoming Request
info: EmployeeManagement.Startup[0]
  MW3: Request handled and response produced
info: EmployeeManagement.Startup[0]
  MW2: Outgoing Response
info: EmployeeManagement.Startup[0]
  MW1: Outgoing Response
```

```
graph LR
  MW1 -- M1 --> MW2
  MW2 -- M2 --> MW3
  MW3 -- M3 --> MW2
  MW2 -- M2 --> MW1
  MW1 -- M1 --> MW2
```

Имаме 3 MiddleWare компонента в pipeline, като NEXT играе ключова роля в прескачането.

Ако наложим още next в края на всеки Use

```
app.Use(async (context, next) =>
{
  // await context.Response.WriteAsync("Hello from 1-st Middleware!");
  logger.LogInformation("MW1: Incoming Request");
  await next();
  logger.LogInformation("MW1: Outgoing Response");
  await next();
});

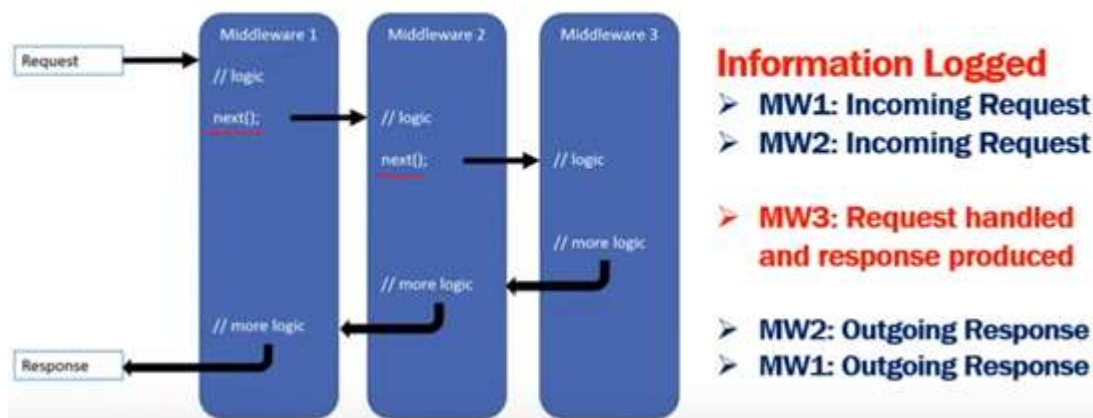
app.Use(async (context, next) =>
{
  logger.LogInformation("MW2: Incoming Request");
  await next();
  logger.LogInformation("MW2: Outgoing Response");
  await next();
});
```

ще се получава много циклещ



```
✓ TERMINAL
    MW1: Incoming Request
info: EmployeeManagement.Startup[0] 1, 2, 3
    MW2: Incoming Request
info: EmployeeManagement.Startup[0]
    MW3: Request handled and response produced
info: EmployeeManagement.Startup[0]
    MW2: Outgoing Response 2, 3
info: EmployeeManagement.Startup[0]
    MW3: Request handled and response produced
info: EmployeeManagement.Startup[0]
    MW1: Outgoing Response 1, 2, 3
    MW2: Incoming Request
info: EmployeeManagement.Startup[0]
    MW3: Request handled and response produced
info: EmployeeManagement.Startup[0]
    MW2: Outgoing Response 2, 3
info: EmployeeManagement.Startup[0]
    MW3: Request handled and response produced
info: EmployeeManagement.Startup[0]
    MW1: Incoming Request 1, 2, 3
    MW2: Incoming Request
info: EmployeeManagement.Startup[0]
    MW3: Request handled and response produced
info: EmployeeManagement.Startup[0]
    MW2: Outgoing Response 2, 3
info: EmployeeManagement.Startup[0]
    MW3: Request handled and response produced
```

Configure Request Processing Pipeline



Everything that happens **before the next() method** is invoked in each of the middleware components, happens as the **REQUEST** travels from middleware to middleware through the pipeline and this is represented by the **incoming arrow**

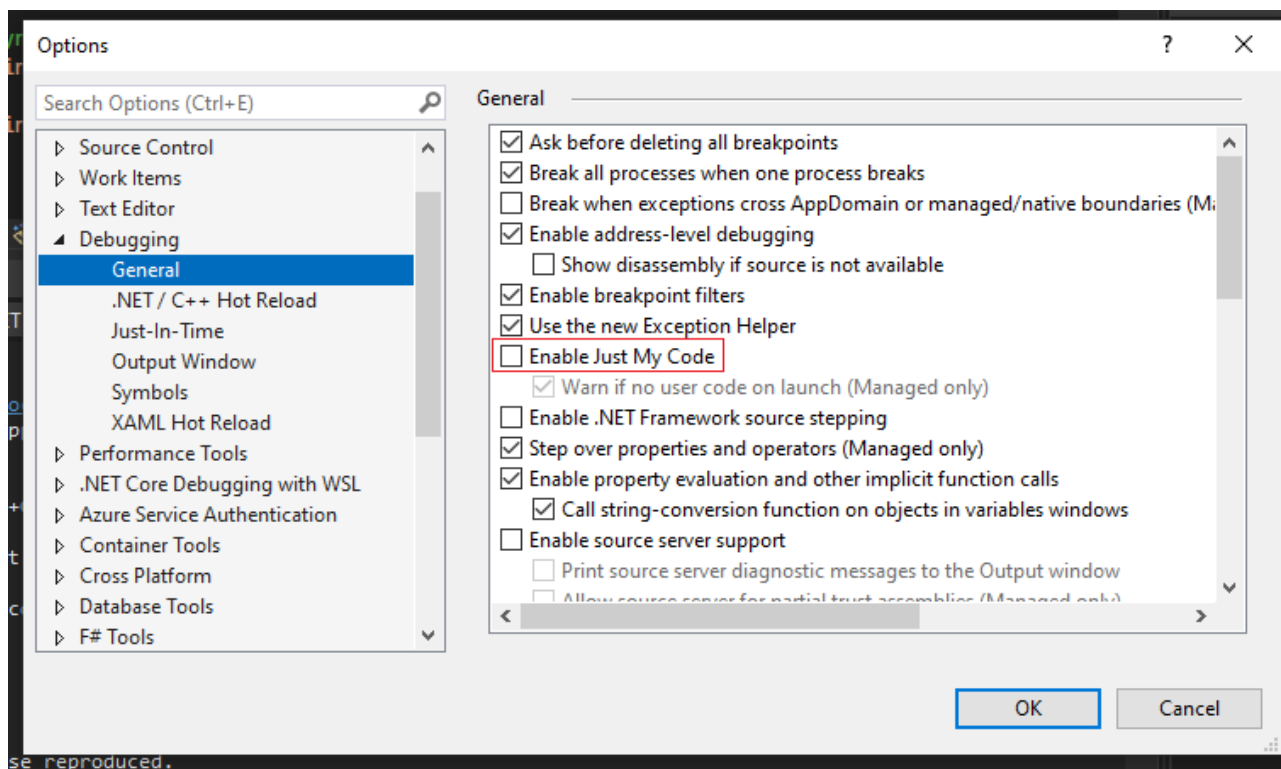
When a middleware handles the request and produces response, the request processing pipeline **starts to reverse**

Everything that happens **after the next() method** is invoked in each of the middleware components, happens as the **RESPONSE** travels from middleware to middleware through the pipeline and this is represented by the **outgoing arrow**

Възможно е, вместо изписаните проследяващи информативни съобщения, да пише само „Just My Code”.

```
debugger option 'Just My Code' is enabled.  
the debugger option 'Just My Code' is enabled.  
imized and the debugger option 'Just My Code' is enabled.  
zed and the debugger option 'Just My Code' is enabled.  
option 'Just My Code' is enabled.  
bugger option 'Just My Code' is enabled.  
and the debugger option 'Just My Code' is enabled.
```

“Just My Code” се определя от Debug -> Options с отметката на Enable за съобщението.



Закоментираме с многоредов коментар `/* */` кода с двата Use, т.к. вече няма да ни трябва

```
/*
    app.Use(async (context, next) =>
    {
        // await context.Response.WriteAsync("Hello from 1-st Middleware!");
        logger.LogInformation("MW1: Incoming Request");
        await next();
        logger.LogInformation("MW1: Outgoing Response");
    });

    app.Use(async (context, next) =>
    {
        logger.LogInformation("MW2: Incoming Request");
        await next();
        logger.LogInformation("MW2: Outgoing Response");
    });
*/
```

И си оставяме само едно “Hello world!”

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("Hello world!");
        // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
        // await context.Response.WriteAsync(_config ["MyKey"]);
        // await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
        //await context.Response.WriteAsync("MW3");
        //logger.LogInformation("MW3: Request handled and response produced");
    });
});
```